Diseño estudiantes

Diana Gabriela Rodríguez

El sistema recibe una única entrada la cual contiene los nombres y promedios de n estudiantes, estos separados por espacios, verifica quienes tienes notas mayor o igual a 3 y al final muestra la lista de aprobados

Requerimientos

1. Entrada: Cadena con nombre y nota, estos separados por espacios
2. . split: para separar los datos de la lista
3. Lista: los nombres se guardan como texto y las notas como float (números reales, incluyen decimales)
4. Divide y conquista: función recursiva
5. Caso base: que al menos la lista tenga un estudiante
6. Evaluar: aprueba si la nota es igual o mayor a 3
7. Caso base dividido: si no es el caso base, el sistema lo subdive en listas
8. Combinar: combina los resultados de las llamadas recursivas en una lista
9. Aprobados: mostrar la lista de aprobados

Historias de usuario

H01- ingresar datos:

Ingrese una cadena con nombres y notas para que el sistema lea la información

- Entrada: cadena de texto
- Salida: lista de datos separados
- Éxito: los datos son ingresados en pares (nombre nota)
- Fracaso: cadena vacía o estructura invalida

H02-divide y conquista

El sistema se encarga de convertir la entrada en una lista nombre(texto), nota(float)

- Entrada: lista de datos
- Lista: alternación de nombre y nota, sin comas en ningún lugar
- Organización: lista organizada
- Éxito: notas bien digitadas sin comas o puntos, solo entre números para la parte decimal
- Fracaso: nota invalida, datos no completos

H03-filtro aprobados

El sistema se encarga de la división recursiva de la lista y verificar quien tienen nota mayor o igual a 3

- Entrada: lista alternada (nombre nota)
- Salida: lista de aprobados
- Éxito: se llega al caso base y se evalúa al estudiante
- Fracaso: la lista no tiene los pares completos

H04-unificación

El sistema se encarga de unir los aprobados encontrados en cada mitad para hacer efectiva la función dividir y conquistar

- Entrada: lista parcial de aprobados
- Salida: lista final de aprobados
- Existo: se combinan los resultados con éxito
- Fracaso: lista vacía

H05-resultado

El sistema imprime los nombres de los aprobados

- Entrada: lista aprobados
- Salida: nombres de los aprobados
- Éxito: se muestran todos los aprobados
- Fracaso: la lista es vacía y no arroja ningún resultado

Diagrama de secuencia

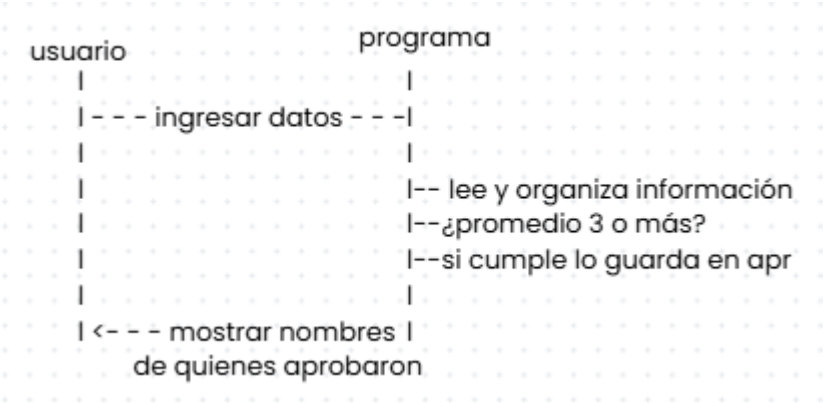


Diagrama caso de uso

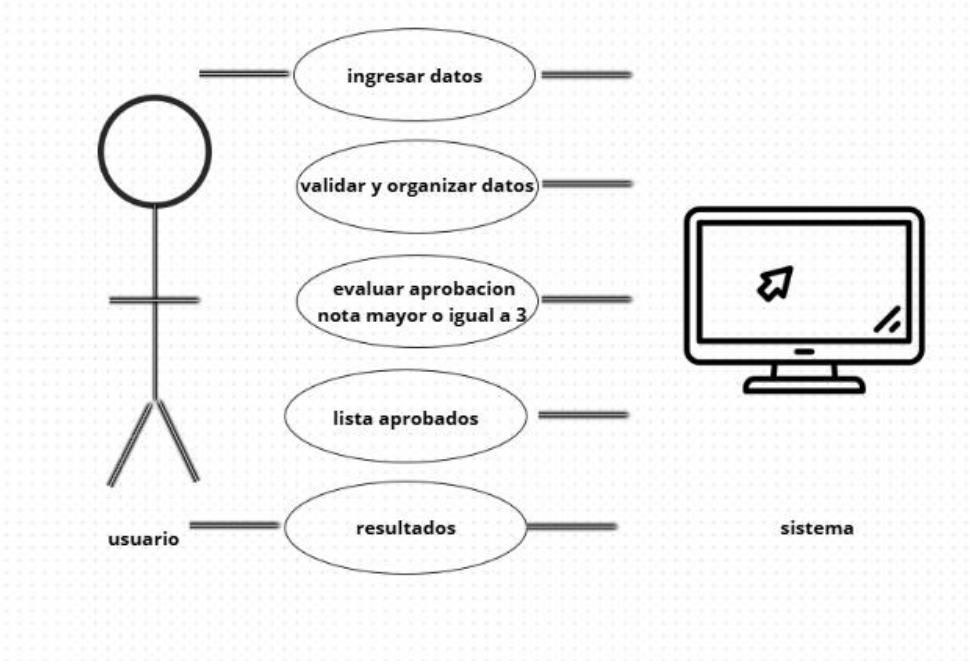
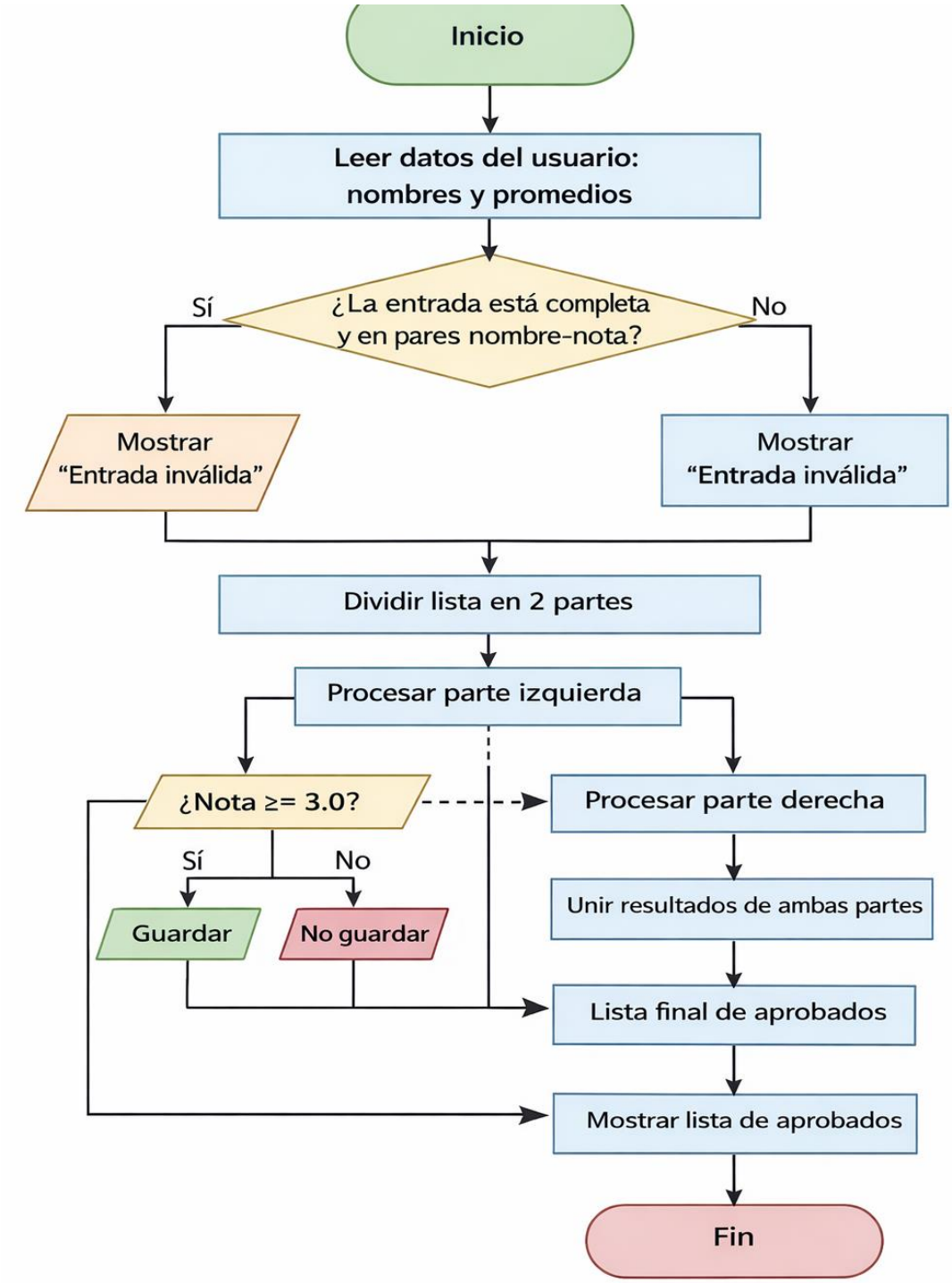


diagrama de flujo de datos



Función recursiva:

$$T(n) = 2T\left(\frac{n}{2}\right) + F(n)$$
$$a = 2, b = 2, F(n) = 1$$
$$T(n) = O(n)$$

función main: T(n)= $O(n)$

Mejor caso y peor caso = O(n) lineal

COST	**TIME**
C1	1
C2	1
C3	1
C4	1
C5	**T(n/2) + n/2**
C6	**T(n/2) + n/2**
C7	**n**
C8	**1**
C9	**1**
C10	**n**
C11	**n**
C12	**n**
C13	**T(n)**
C14	**k**

```python
def aprobar(datos):
    if len(datos)== 2:
        return [datos[0]] if datos[1] >= 3.0 else []
    mitad= len(datos)//2
    mitad= mitad- (mitad%2)
    izquierda= aprobar(datos[:mitad])
    derecha= aprobar(datos[mitad:])
    return izquierda + derecha
def main():
    lista= input("ingrese nombre y nota separados por espacios: ").split()
    datos= []
    for i in range(0, len(lista), 2):
        datos.append(lista[i])
        datos.append(float(lista[i + 1]))
    apr= aprobar(datos)
    print("Los estudiantes que pasaron son:", apr)

main()
```

Análisis

En el algoritmo se utilizó **divide y conquista**, dividiendo la lista de estudiantes en partes más pequeñas hasta llegar al caso base (un solo estudiante). Luego, los resultados se combinan para obtener la lista final de aprobados.

La complejidad del algoritmo es **O(n log n)**, ya que en cada nivel se divide el problema en dos partes y se realiza un trabajo proporcional al tamaño de la lista.

Esto significa que el tiempo de ejecución crece de manera eficiente a medida que aumenta la cantidad de estudiantes.